# Lab 3 - Groub 10 - Symmetric Encryption: Block Ciphers, IND-CPA, AEAD & Cryptographic Failures

**Course:** CSCI/CSCY 4407 — Security & Cryptography **Semester:** Spring 2026 **Date:** 02/27/26 **Group Members:** Cassius Kemp, Matthew Kenner, Jonathan Le

---

## Task 1 — ECB Distinguisher (15 pts)

### Source Code

```
"""
Task 1 — ECB Distinguisher (15 pts)
===================================
Demonstrates that AES-ECB is deterministic and fails IND-CPA.

Approach:
- P0: repeated 16-byte block (same block N times)
- P1: random bytes of the same length
- Encrypt both under AES-ECB with a random key
- Distinguisher counts duplicate 16-byte ciphertext blocks
- Run >= 20 trials, record success rate and IND-CPA advantage
"""

import os
import random
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

BLOCK_SIZE = 16
NUM_BLOCKS = 8       # plaintext length = NUM_BLOCKS * 16 bytes
NUM_TRIALS = 20


# ---------------------------------------------------------------------------
# AES-ECB helpers
# ---------------------------------------------------------------------------

def generate_key() -> bytes:
    """Generate a random 256-bit AES key."""
    return os.urandom(32)


def ecb_encrypt(key: bytes, plaintext: bytes) -> bytes:
    """Encrypt plaintext under AES-ECB (plaintext must be block-aligned)."""
    cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=default_backend())
    encryptor = cipher.encryptor()
    return encryptor.update(plaintext) + encryptor.finalize()


# ---------------------------------------------------------------------------
# Plaintext generation
# ---------------------------------------------------------------------------

def make_p0(num_blocks: int = NUM_BLOCKS) -> bytes:
    """P0: the same 16-byte block repeated num_blocks times."""
    block = os.urandom(BLOCK_SIZE)
    return block * num_blocks
```

```python
def make_p1(num_blocks: int = NUM_BLOCKS) -> bytes:
    """P1: num_blocks of independent random 16-byte blocks."""
    return os.urandom(num_blocks * BLOCK_SIZE)


# ---------------------------------------------------------------------------
# Distinguisher
# ---------------------------------------------------------------------------

def count_duplicate_blocks(ciphertext: bytes) -> int:
    """Return the number of repeated 16-byte blocks in ciphertext."""
    blocks = [ciphertext[i:i + BLOCK_SIZE] for i in range(0, len(ciphertext), BLOCK_SIZE)]
    return len(blocks) - len(set(blocks))


def distinguisher(ciphertext: bytes) -> int:
    """
    Guess b' based on ciphertext structure.
    Returns 0 if repeated blocks detected (guess: P0 was encrypted),
    Returns 1 otherwise (guess: P1 was encrypted).
    """
    if count_duplicate_blocks(ciphertext) > 0:
        return 0   # b' = 0  (looks like repeated plaintext)
    return 1       # b' = 1  (looks like random plaintext)


# ---------------------------------------------------------------------------
# IND-CPA game
# ---------------------------------------------------------------------------

def run_trial(key: bytes) -> bool:
    """
    One trial of the IND-CPA experiment.
    Choose b randomly, encrypt P_b, run distinguisher, return whether b' == b.
    """
    b = random.randint(0, 1)
    plaintext = make_p0() if b == 0 else make_p1()
    ciphertext = ecb_encrypt(key, plaintext)
    b_prime = distinguisher(ciphertext)
    return b_prime == b


def run_experiment(num_trials: int = NUM_TRIALS) -> None:
    key = generate_key()
    print(f"AES key (hex): {key.hex()}")
    print(f"Plaintext length: {NUM_BLOCKS * BLOCK_SIZE} bytes ({NUM_BLOCKS} blocks)\n")

    # Show an example of P0 vs P1 encryption to illustrate block repetition
    p0 = make_p0()
    p1 = make_p1()
    c0 = ecb_encrypt(key, p0)
    c1 = ecb_encrypt(key, p1)
    print("=== Example ciphertexts ===")
    print(f"C0 (from repeated P0): {c0.hex()}")
    print(f"  duplicate blocks: {count_duplicate_blocks(c0)}")
    print(f"C1 (from random   P1): {c1.hex()}")
    print(f"  duplicate blocks: {count_duplicate_blocks(c1)}\n")

    # Run trials
    print(f"{'Trial':<8} {'b':<6} {'b_prime':<10} {'Correct'}")
    print("-" * 35)
    correct = 0
    for i in range(1, num_trials + 1):
        b = random.randint(0, 1)
        plaintext = make_p0() if b == 0 else make_p1()
        ciphertext = ecb_encrypt(key, plaintext)
```

```
        b_prime = distinguisher(ciphertext)
        result = b_prime == b
        if result:
            correct += 1
        print(f"{i:<8} {b:<6} {b_prime:<10} {'Yes' if result else 'No'}")

    success_rate = correct / num_trials
    advantage = abs(success_rate - 0.5)
    print(f"\nCorrect: {correct}/{num_trials}")
    print(f"Success rate: {success_rate:.2%}")
    print(f"Adv_IND-CPA = |Pr[b'=b] - 1/2| = {advantage:.4f}")


if __name__ == "__main__":
    run_experiment()
```

## Repeated Ciphertext Block Evidence

Ciphertext output showing duplicate 16-byte blocks for P0 (repeated plaintext)

## Trial Results Table (≥ 20 Trials)

```
└─$ python3 task1_ecb_distinguisher.py
AES key (hex): 37189b6f2bf22e38d2bc4de53aa50d57d18ca8b7dbaf230d4b9d03e561de8e
ea
Plaintext length: 128 bytes (8 blocks)

═══ Example ciphertexts ═══
C0 (from repeated P0): 865ad50d252ca9acb8914b11af497c65865ad50d252ca9acb8914b
11af497c65865ad50d252ca9acb8914b11af497c65865ad50d252ca9acb8914b11af497c65865
ad50d252ca9acb8914b11af497c65865ad50d252ca9acb8914b11af497c65865ad50d252ca9ac
b8914b11af497c65865ad50d252ca9acb8914b11af497c65
  duplicate blocks: 7
C1 (from random   P1): 6b6d8d100f0389d38b39f2f0ddffd0f0d58111e6af5be29e46b489
d62a38bb5f6517199da07a11c62f8623e909e30436f80296a71b361340888e2b4d9f44d390b54
533e918532b08c2b90d248b62443d7ffae35dac978933677e18d450b49dcaed4f96f0120d7bdc
3d27743a979bb00d82fff4a28f5732cf78ce8127a85e75d8
  duplicate blocks: 0

Trial    b      b_prime    Correct
──────────────────────────────────
1        1      1          Yes
2        1      1          Yes
3        1      1          Yes
4        1      1          Yes
5        0      0          Yes
6        1      1          Yes
7        1      1          Yes
8        0      0          Yes
9        1      1          Yes
10       1      1          Yes
11       1      1          Yes
12       1      1          Yes
13       1      1          Yes
14       0      0          Yes
15       0      0          Yes
16       0      0          Yes
17       0      0          Yes
18       0      0          Yes
19       1      1          Yes
20       0      0          Yes

Correct: 20/20
Success rate: 100.00%
Adv_IND-CPA = |Pr[b'=b] - 1/2| = 0.5000
```

## Success Rate and Advantage

**Number of correct guesses:** [20 / 20]

**Success Rate:** [20 / 20 = 100%]

**IND-CPA Advantage:**

`Adv_IND-CPA = |Pr[b' = b] - 1/2| = |1.0 - 0.5| = 0.5`

### Explanation: Why ECB Violates Semantic Security

ECB mode encrypts each 16-byte block independently and deterministically:

```
C_i = E_k(P_i)
```

Because AES-ECB uses no IV/nonce, the same plaintext block always produces the same ciphertext block under the same key. Therefore, if a plaintext contains repeated blocks (P_i = P_j), ECB will produce repeated ciphertext blocks (C_i = C_j). This leaks structural information about the plaintext (e.g., repetitions/patterns) even though the key is secret.

This violates semantic security / IND-CPA because an attacker can distinguish between encryptions of two chosen messages with non-negligible advantage. In our experiment, P0 was constructed with repeated 16-byte blocks, which caused repeated ciphertext blocks after encryption. P1 was random-looking and produced no repeated ciphertext blocks. The distinguisher simply checks for duplicate ciphertext blocks; if duplicates exist it guesses “P0,” otherwise it guesses “P1.” Across 20 trials the distinguisher succeeded 20/20 times, giving Pr[b’=b]=1.0 and an IND-CPA advantage of 0.5, far better than random guessing (0.5). Therefore, ECB is not IND-CPA secure.

---

# Task 2 — CBC IV Experiment (15 pts)

## Source Code

```
"""
Task 2 — CBC IV Experiment (15 pts)
====================================
Demonstrates the effect of IV handling in AES-CBC.

CBC formula: C_i = E_k(P_i XOR C_{i-1}),  C_0 = IV

Experiments:
1. Fresh IV each time  → different ciphertexts (unlinkable)
2. Reused IV           → identical ciphertexts (linkable / deterministic)
3. Verify decryption correctness via SHA-256
"""

import os
import hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend

BLOCK_SIZE = 16
PLAINTEXT_SIZE = 256  # >= 256 bytes as required


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------

def sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()


def pkcs7_pad(data: bytes, block_size: int = BLOCK_SIZE) -> bytes:
    padder = padding.PKCS7(block_size * 8).padder()
    return padder.update(data) + padder.finalize()


def pkcs7_unpad(data: bytes, block_size: int = BLOCK_SIZE) -> bytes:
    unpadder = padding.PKCS7(block_size * 8).unpadder()
    return unpadder.update(data) + unpadder.finalize()


def cbc_encrypt(key: bytes, iv: bytes, plaintext: bytes) -> bytes:
    """Encrypt plaintext with AES-CBC using PKCS#7 padding."""
    padded = pkcs7_pad(plaintext)
```

```
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    return encryptor.update(padded) + encryptor.finalize()


def cbc_decrypt(key: bytes, iv: bytes, ciphertext: bytes) -> bytes:
    """Decrypt ciphertext with AES-CBC and remove PKCS#7 padding."""
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    decryptor = cipher.decryptor()
    padded = decryptor.update(ciphertext) + decryptor.finalize()
    return pkcs7_unpad(padded)


# ---------------------------------------------------------------------------
# Experiments
# ---------------------------------------------------------------------------

def experiment_fresh_iv(key: bytes, plaintext: bytes) -> None:
    """
    Encrypt the same plaintext twice with DIFFERENT random IVs.
    Expected: ciphertexts differ — SHA-256 hashes should not match.
    """
    print("=== Experiment 1: Fresh IV (different each time) ===")
    iv1 = os.urandom(BLOCK_SIZE)
    iv2 = os.urandom(BLOCK_SIZE)
    c1 = cbc_encrypt(key, iv1, plaintext)
    c2 = cbc_encrypt(key, iv2, plaintext)
    h1 = sha256(c1)
    h2 = sha256(c2)
    print(f"  IV1:             {iv1.hex()}")
    print(f"  IV2:             {iv2.hex()}")
    print(f"  SHA-256(C1):     {h1}")
    print(f"  SHA-256(C2):     {h2}")
    print(f"  Ciphertexts match: {h1 == h2}  (expected: False)\n")


def experiment_reused_iv(key: bytes, plaintext: bytes) -> None:
    """
    Encrypt the same plaintext twice with the SAME IV.
    Expected: ciphertexts are identical — SHA-256 hashes must match.
    """
    print("=== Experiment 2: Reused IV (same IV both times) ===")
    iv = os.urandom(BLOCK_SIZE)
    c1 = cbc_encrypt(key, iv, plaintext)
    c2 = cbc_encrypt(key, iv, plaintext)
    h1 = sha256(c1)
    h2 = sha256(c2)
    print(f"  IV:              {iv.hex()}")
    print(f"  SHA-256(C1):     {h1}")
    print(f"  SHA-256(C2):     {h2}")
    print(f"  Ciphertexts match: {h1 == h2}  (expected: True)\n")


def experiment_decryption_verification(key: bytes, plaintext: bytes) -> None:
    """
    Encrypt then decrypt and verify via SHA-256.
    """
    print("=== Experiment 3: Decryption Correctness (SHA-256 verification) ===")
    iv = os.urandom(BLOCK_SIZE)
    ciphertext = cbc_encrypt(key, iv, plaintext)
    recovered = cbc_decrypt(key, iv, ciphertext)
    h_orig = sha256(plaintext)
    h_recv = sha256(recovered)
    print(f"  SHA-256(original):  {h_orig}")
    print(f"  SHA-256(decrypted): {h_recv}")
    print(f"  Match: {h_orig == h_recv}  (expected: True)\n")
```

```
# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    key = os.urandom(32)
    plaintext = os.urandom(PLAINTEXT_SIZE)

    print(f"AES key (hex): {key.hex()}")
    print(f"Plaintext length: {len(plaintext)} bytes\n")

    experiment_fresh_iv(key, plaintext)
    experiment_reused_iv(key, plaintext)
    experiment_decryption_verification(key, plaintext)
```

### Evidence: Fresh IV → Different Ciphertext Experiment 1

### Evidence: Reused IV → Linkability

### Decryption Verification (SHA-256)

```
┌──(kali㉿kali)-[/media/sf_Security&Crypto/CSCI-4407-lab3/scripts]
└─$ python3 task2_cbc_iv.py
AES key (hex): 6af93862e5b5ac721384f71fe81fca04c645d1d81aa9902bee0c1df60e31725e
Plaintext length: 256 bytes

=== Experiment 1: Fresh IV (different each time) ===
  IV1:              7db7def6e27eeee8a603069474b32e7a
  IV2:              4752d82023a3d25c577923324d35a9c6
  SHA-256(C1):      e69fde9ee50368206f71a82a7ea2c0d088afe9a3412500593d2585e4e0a24cfe
  SHA-256(C2):      6dc977078604acc18185a0959d639dba90a25c8df8ae6a934159bc156bd057a6
  Ciphertexts match: False  (expected: False)

=== Experiment 2: Reused IV (same IV both times) ===
  IV:               9c5173ca435e90af6e337fa7f3cf4a18
  SHA-256(C1):      2ac0e46253ad52fd4c37b045d1a3413c5438ea641de0a9e9c8cff006eac2e048
  SHA-256(C2):      2ac0e46253ad52fd4c37b045d1a3413c5438ea641de0a9e9c8cff006eac2e048
  Ciphertexts match: True  (expected: True)

=== Experiment 3: Decryption Correctness (SHA-256 verification) ===
  SHA-256(original):  35dce44982fa3cf6a9017b2132ebab5930aca52a8cc641474db7f0de144e1d47
  SHA-256(decrypted): 35dce44982fa3cf6a9017b2132ebab5930aca52a8cc641474db7f0de144e1d47
  Match: True  (expected: True)
```

### Explanation: Why a Fresh IV Is Required

CBC mode uses an initialization vector (IV) in the first block:

```
C1 = E_k(P1 ⊕ IV)
Ci = E_k(Pi ⊕ C_{i-1}) for i ≥ 2
```

The IV must be fresh (unique and unpredictable) for each encryption under the same key. When we encrypted the same plaintext twice with two different random IVs, the ciphertexts differed (their SHA-256 hashes were different), which is expected because changing the IV changes the input to the first block and therefore changes the entire chained ciphertext.

When we reused the same IV to encrypt the same plaintext twice, we obtained identical ciphertexts (matching SHA-256 hashes). This creates linkability: an observer can tell that the same message (or at least the same first block) was encrypted again under the same key/IV pair. Therefore, IV reuse undermines semantic security in CBC by leaking relationships between messages. Using a fresh random IV each time prevents this linkability and is required for CBC to achieve IND-CPA-style confidentiality.

---

## Task 3 — CTR Implementation (10 pts)

### Source Code

```
import os
```

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

def GetAESCTRCipher(key, nonce):
    #AES-CTR needs a 16-byte nonce (This functions as our initial counter block)
    return Cipher(algorithms.AES(key), modes.CTR(nonce), backend=default_backend())

def FileEncryption(input_file, output_file, key, nonce):
    Cipher = GetAESCTRCipher(key, nonce)
    Encryptor = Cipher.encryptor()

    with open(input_file, 'rb') as f_in:
        Plaintext = f_in.read()

    CipherText = Encryptor.update(Plaintext) + Encryptor.finalize()

    with open(output_file, 'wb') as f_out:
        f_out.write(CipherText)

def FileDecryption(input_file, output_file, key, nonce):
    Cipher = GetAESCTRCipher(key, nonce)
    Decryptor = Cipher.decryptor()

    with open(input_file, 'rb') as f_in:
        CipherText = f_in.read()

    PlainText = Decryptor.update(CipherText) + Decryptor.finalize()

    with open(output_file, 'wb') as f_out:
        f_out.write(PlainText)

if __name__ == "__main__":
    #Generates a Key (32 bytes for AES-256) and a Nonce (16 bytes)
    key = os.urandom(32)
    nonce = os.urandom(16)

    print(f"Key (hex): {key.hex()}")
    print(f"Nonce (hex): {nonce.hex()}")

    #Encrypts
    print("Encrypting ctr_msg.bin \n Creating ctr_ct.bin...")
    FileEncryption("ctr_msg.bin", "ctr_ct.bin", key, nonce)

    #Decrypts
    print("Decrypting ctr_ct.bin... \n Creating ctr_recovered.bin...")
    FileDecryption("ctr_ct.bin", "ctr_recovered.bin", key, nonce)

    print("Done.")
```

## Key and Nonce Details

| Parameter | Size | Value |
|---|---|---|
| Key | 32 bytes | a9de4d8ea97c8664fafe81432d4ecba8e6a73711c2978fb5e49d7fc31fd64816 |
| Nonce | 16 bytes | 05bc786abd24e5781d8b83002820ff4d |
| Test file | 4096 bytes | — |

### Encryption and Decryption Output

```
┌──(kali㉿kali)-[~]
└─$ cd ~/Lab3/Task3

┌──(kali㉿kali)-[~/Lab3/Task3]
└─$ head -c 4096 /dev/urandom > ctr_msg.bin

┌──(kali㉿kali)-[~/Lab3/Task3]
└─$ python3 CTR.py
Key (hex): a9de4d8ea97c8664fafe81432d4ecba8e6a73711c2978fb5e49d7fc31fd64816
Nonce (hex): 05bc786abd24e5781d8b83002820ff4d
Encrypting ctr_msg.bin
 Creating ctr_ct.bin ...
Decrypting ctr_ct.bin ...
 Creating ctr_recovered.bin ...
Done.

┌──(kali㉿kali)-[~/Lab3/Task3]
└─$ sha256sum ctr_msg.bin ctr_recovered.bin
767f64063b2843f478f0b1763d97c9052e0023e2bcd7b0de9a473cfcfd6b9ef8  ctr_msg.bin
767f64063b2843f478f0b1763d97c9052e0023e2bcd7b0de9a473cfcfd6b9ef8  ctr_recovered.bin
```

### SHA-256 Hash Verification

```
SHA-256(original file):   767f64063b2843f478f0b1763d97c9052e0023e2bcd7b0de9a473cfcfd6b9ef8
SHA-256(decrypted file):  767f64063b2843f478f0b1763d97c9052e0023e2bcd7b0de9a473cfcfd6b9ef8
Match: Yes
```

### Explanation: Why CTR Behaves Like a Stream Cipher

AES-CTR transforms a block cipher into a stream cipher; instead of encrypting the plaintext directly the block cipher encrypts a unique input block consisting of a Nonce and a Counter (IV=Nonce||Counteri). This produces a stream of pseudorandom bits known as the keystream, the encryption process is the XOR between the plaintext and the keystream: Ci=Pi⊕Ek(Nonce||Counteri) Because the XOR operation is symmetric decryption is identical to encryption. The receiver generates the same keystream and XORs with the ciphertext to recover the plaintext, unlike the CBC mode there is no chaining between the blocks. This allows for parallel processing meaning that multiple blocks can be encrypted or decrypted simultaneously, which is a just how stream ciphers operate.

---

## Task 4 — CTR Nonce Reuse Attack (20 pts)

### Group Ciphertext Package

**Group number:** 10 **Files used:** [LIST FILENAMES FROM GROUP PACKAGE]

### XOR Script

```
import sys

def XORByter(b1, b2):
    """XORs two strings together, and then returns bytes."""
    #We use zip to XOR byte by byte, then the length will be the shorter of the two.
    return bytes([x ^ y for x, y in zip(b1, b2)])

def HexExtractionPreview(data, length=64):
    """Prints a hexdump-style preview of the data (Hex + ASCII)."""
    print(f"\n--- PREVIEW OF FIRST {length} BYTES ---")
    print("OFFSET  HEX BYTES                                ASCII REPRESENTATION")
    print("-" * 70)

    # Take only the first 'length' bytes
    chunk = data[:length]

    # Process in rows of 16 bytes
    for i in range(0, len(chunk), 16):
        row = chunk[i:i+16]

        # Hex section
```

```
        hex_vals = ' '.join(f"{b:02x}" for b in row)

        # ASCII section (replace unprintable chars with '.')
        ascii_vals = ''.join(chr(b) if 32 <= b <= 126 else '.' for b in row)

        # Print formatted row
        print(f"{i:04x}    {hex_vals:<48}  |{ascii_vals}|")
    print("-" * 70)

def main():
    print("--- CTR Nonce Reuse Attack ---")

    #Loads the file provided C1, C2, P1
    try:
        print("Loading files...")
        with open("C1.bin", "rb") as f:
            C1 = f.read()
        with open("C2.bin", "rb") as f:
            C2 = f.read()
        with open("P1.bin", "rb") as f:
            P1 = f.read()

        print(f"    Loaded C1: {len(C1)} bytes")
        print(f"    Loaded C2: {len(C2)} bytes")
        print(f"    Loaded P1: {len(P1)} bytes")

    except FileNotFoundError as e:
        print(f"\n Error: {e}")
        print("Make sure C1.bin, C2.bin, and P1.bin are in this folder.")
        return

    #Verify the string lengths
    MinimumLength = min(len(C1), len(C2), len(P1))

    #Trim to the minimum length to avoid overflow
    C1 = C1[:MinimumLength]
    C2 = C2[:MinimumLength]
    P1 = P1[:MinimumLength]

    #Attack
    print("\nCalculating Keystream...")
    KeyStream = XORByter(C1, P1)

    print("Recovering P2...")
    P2 = XORByter(C2, KeyStream)

    hexdump_preview(P2)

    #Check for common file signatures automatically
    if P2.startswith(b'%PDF'):
        print("[!] DETECTED PDF FILE HEADER!")
    elif P2.startswith(b'\x89PNG'):
        print("[!] DETECTED PNG IMAGE HEADER!")
    elif P2.startswith(b'\xff\xd8\xff'):
        print("[!] DETECTED JPEG IMAGE HEADER!")
    else:
        print("No file header found. Assumed that plain text has been found.")

    #Display results
    print("\n--- RECOVERED PLAINTEXT ---")
    try:
        #Print as text
        print(P2.decode('utf-8'))
    except UnicodeDecodeError:
        #If it contains non-printable characters show them in hex instead
        print("Output contains non-printable characters. Showing Hex:")
```

```
        print(P2.hex())

    #Save the result
    with open("P2_recovered.txt", "wb") as f:
        f.write(P2)
    print("\n Saved recovered message to 'P2_recovered.txt'")

if __name__ == "__main__":
    main()
```

## XOR Output Evidence

```
┌──(kali㉿kali)-[~/Lab3/Task4]
└─$ python3 attacker.py
── CTR Nonce Reuse Attack ──
Loading files ...
    Loaded C1: 4096 bytes
    Loaded C2: 4096 bytes
    Loaded P1: 4096 bytes

Calculating Keystream ...
Recovering P2 ...

── PREVIEW OF FIRST 64 BYTES ──
OFFSET  HEX BYTES                                       ASCII REPRESENTATION
──────────────────────────────────────────────────────────────────────────────
0000    43 4f 4e 46 49 44 45 4e 54 49 41 4c 20 4d 45 4d   |CONFIDENTIAL MEM|
0010    4f 3a 20 44 6f 20 6e 6f 74 20 73 68 61 72 65 2e   |O: Do not share.|
0020    20 50 72 6f 6a 65 63 74 3d 53 59 4d 4d 45 54 52   | Project=SYMMETR|
0030    49 43 2d 4c 41 42 3b 20 4d 6f 64 65 3d 43 54 52   |IC-LAB; Mode=CTR|
──────────────────────────────────────────────────────────────────────────────
No file header found. Assumed that plain text has been found.
```

```
── RECOVERED PLAINTEXT ──
Output contains non-printable characters. Showing Hex:
434f4e464944454e5449414c204d454d4f3a20446f206e6f742073686172652e2050726f6a6563743d53594d4d45545249432d4c41423b204d6f64653d435452cb59f7d76b
8ee461848a1320deebf1bfc66d0fcd7df1f4ab8955c35bdf90d3a3d1a33b4656b787f0953b82a37d8c8f8f3f37705b57d37b4b735992f69a5df003fd03cd896cfce2f837af
d179b968917253225a250dfcce36a1682ba684cd5dc96361306cc3904b4f9b468cd93851ec7b1236c6b29f758f2501abf734a35a04cd5c3ec262463bf0963a7d69029f27cc
57eb6f1353b21482cd673f040a00b46f24bea9be4dadbe077a7114f94faf27acb971531e917bf993111e3ce1882570874d2e1d8bb17dfb2c8a3b80b58b497e7a3eb9438b70
4c8a22c3c6d4d04d392d92d60e93258f623a4f088b78a13efe22af03ecc5be681ebfd9cc0b89a074932b6b99a5d9779acf3dda3ad26085d88a457f72af003f0b125f24e9e8
d137c1a22632d4b6add41dc6a0d72e3bf5bc1296ba42d0ea869cf2d3d73ee29ae6a43adc5536cca43a590060e6bb6a764d93568f64e0d68c056ddcad80bc855a2154ba743e
db1d61ca21f00ec1f0598f6f55d25286fc336120ddb8606957c30042a8d48d5d73252c61b66299241ffcdd1921be51594500bd949dfa7014b9b6877d54694812c3edf9d11f
4c4537d14622f6c300e6b66792e546ddcea4cc6a7e4b5451747b4d61318b8dc06a4cefa42f64029ef99ab450c20a3de0678a3bbaf122c79ab76083ea2025aed1d1c20996b8
b2d8ea8e52095a094b1ebf0b57a43ba45938c92417b3fd479832cc62eeefcc110658d561b3263b59f7fccf7eb57f3e546fd408f354b244342b79c9e5d643a3023d4d33e385
2f93ef22fb1851755b0a2427d5aaf49f9b0ca22b530a604bee4d5ffdfa8530f0aa49a74401bcb75551dbc628f75946831a6343fc2d04880781cfbb457227d99a8a9375bc33
c27809397494bcc997b1a44b0ae2de93e4426749c25a5fcf109673bf734cb8e09c90e8fce3ed1844b12d7788ebddaf76ead5c88b36f1e7e9b18e778b4023c5799c6da111c1
b4c9d512ce12a48f83644f63c22cb49bfea8f9884bc96a09818966239d774747e48a3dcfa07b39f80282450d2038582dcd7343158d92f65e66c01b701937ba01e95007309a
96e2b586d53b5001425c1e3aa59639b126e38cfd96e363f48d940f1b8c695c91c885691e86686af0fffb2c48c98b49e80b5a3f8ec7edc29a07cdc02df083a211378deef684
e704adee27c112b1ee3bb0d90149dd1fe33465a7095c47216cc80815bfb501a78db5b5911b75efebdc48b0ba396c4be8b779fc47bb08959dfb1a28be8e1c97837afbb3c507
df9989b0160c8c980aa03c77423f8d876a1af88b36775d125619c1c8cf6c6bee0ebf6d82ae4d1243522f320ea540efcb8da2cf1cf1958065caeca74eee9d0520e451c0cd4a
53525d2549a40b4ee774dee951cfb77240c743eaada41d247b2e6d5deeceaeb9b0a8b1c666c98c83fd7a28a0b8b966030e38434e234cd53015873f62a1680f066f886a3926
219cd5257e4eab55706f997cf91d826ebf954e9fbfbd5a768fb3878de7ff771e3737e5c12295663a8e20c853666a7229ee4a608b806223456a9eb170ca1724dbcbed8617c9
117b09fdaa967b53de87aeb94131b1e731b8fb27b043b7a329ff90bd7491974b10dd098834433ab0aaf533969ecd1ff6af6bea4de2a6fc1172b65b01989aa23c8d0701e0a7
e6cb718f08296ce53bf788acdf5ae30a4c76e59f2e210358cdd289e39fc8e492aba5430cdacaf34f2369217b59c4bb9d2ddbe8134be949571a73d7a1e2fe450a49612cfe64
8264e819e9665ff7149325bbf89a7315b5974bce7da2f46372b529e36a0ead3d1b2fee0bd9451d58056656646a58a89d4ff7e339ffd7b194291575edddd7cbfd4ed062b0ca
a6a04fc149e251dbf62301862af08142a11eb3627b3d7963468ac723160f0b4e2023b9913dff5a0f4e7c6495288a36c270e42fcd77efec9e8d3a44e631a63e5aa8bade6f61
ec0cb5bcd22b1c91731b193288cdd6364f8ccbdda8d27e4bc7048368383c09872bf5a3d794d912ac0349c1a317c07c63c5ce6d5e70d5660e99ca4855396b868e54b39bd318
4547e2026b3596b8f5282eef93b2e18f8c0bbc1cc9a94f143e219b8313dec259f14115c4ce6b504a49a38a3a6e120ebd5ca1d125564b411adb6f8639c2b85a4d161b0f915e
89cfaabe7c00316e606f65cf519068045fcfd1e8fca797b6e870b4feaf3b7a835f69fba7f1a27b0eed39d90a6c65f5ca52787af8890202af1dacb04d2fb303b7731d70f61f
b95504dee8dfb5441f6b5e965788f75c4e121e9c2cba8c55ce5acf430b7231634e766b0d6768f87d299f1c85f3f5f313cdf82e37d21ac6637877bb530a965da956793dd405
e2247fbbced2a9b3e267cd234e18b3d37c8fb1bedef557aa93944d877e450330194d035b1d31230a9863bf10756a87b4a4a9ca8fbd1e5dd94c861ca4a55137232193fd3487
71ebfa9cc0c9f76742dc018666f76136ec630c2c09b610ec1c46b0ff44c34edfde0898d0389ade46d0e3088482fccbd770cd8a176dc9bc9ffa65338d4db9997b9676a4945a
1227d49830a0921deb8923a2d509cc750e9a96cbbcb73d816b999f3fd2a44554d514cff80f64e737ec67488fc8db4361b815f355ce17ed539f0d181664e8d2ca59a0eb48af
8b7786c7a82add9ea366df74e1adbdf40faac53e822d10104305f2b77051ebb5e600f97e560fe5677afa3e902e0e35ab335634bfb56b33a6f06a34b9cfe385ced9875d6e37
a2a4298143a26911a47be8e3695e15e441af89fd5b40540350ab7e5cd64cce55506444c73da96b800d02f919f08e9cf274eb989c5a71b40801c61fd42196f75d5dadf618d7
5b1e2ad1f95fe60f04989d6eb993002b047405adfcfb87e35d373a984820d78acf8d18e3bc8c94fe0587892a57ea76dc39077c6b5a1431d6c93a4a0c645d25bf222034ec06
5432f45f907d3aef2a831b408ac99fc2b8c04acc615ff8233afa2261e4ffe30bfba3a8b4a090bbd8c1f8f8440fe95bf2a68ae0f8e44d9155881c045764c564cd9d3db52175
b0efa9b2204be43959f398c7c9595a5565b33a41ff439032b1f37afd064d2ddf6ff9c8b10126136575d4801278244db2c653faa0065c3fa77ba0e2b824c4f0695bc8aedd57
0d984b0c983ce3a5f65a2b9410a5cd6cd42f8bcee23007d1930b4860cc7baa228e150d9c325061f8de961ce348837ee447ba48e943ccf1d717be9ebee409995914cc2b66b2
e21d32ac4b2eee385fc57d75f0a0de25dd30b7f3c0436e676d566af3cecae8063b61a26bb9af1ae71c5c2e433e6089476db0806d9e4c566ee70494548872ba1b7d524aa5fa

 Saved recovered message to 'P2_recovered.txt'
```

## Recovered Plaintext (of readable text at top of P2 file)

| Offset | Hex Bytes | Recovered Text |
|---|---|---|
| 0000 | 43 4f 4e 46 49 44 45 4e 54 49 41 4c 20 4d 45 4d | CONFIDENTIAL MEM |
| 0010 | 4f 3a 20 44 6f 20 6e 6f 74 20 73 68 61 72 65 2e | o: Do not share. |
| 0020 | 20 50 72 6f 6a 65 63 74 3d 53 59 4d 4d 45 54 52 | Project=SYMMETR |

| Offset | Hex Bytes | Recovered Text |
|---|---|---|
| 0030 | 49 43 2d 4c 41 42 3b 20 4d 6f 64 65 3d 43 54 52 | IC-LAB; Mode=CTR |

### Explanation of Keystream Reuse

**Violated assumption: nonce uniqueness** — CTR mode is secure only when every (key, nonce) pair is used for exactly one encryption; reusing the same nonce with the same key directly violates this assumption and renders the scheme insecure.
When a nonce is reused with the same key the input to the encryption is identical. Since AES is a deterministic function it produces the exact same output (the keystream) for both messages. When two different plaintexts (P1 and P2) are encrypted using this identical keystream, the security falls in on it self. If an attacker captures both ciphertexts (C1 and C2) and XORs them together, the keystream cancels out entirely: $C1 \oplus C2 = (P1 \oplus K) \oplus (P2 \oplus K) = P1 \oplus P2 \oplus (K \oplus K) = P1 \oplus P2$. This leaves the attacker with the XOR of the two plaintexts, effectively removing the encryption layer. This creates a classic “Two-Time Pad” vulnerability, where known-plaintext attacks or statistical analysis (crib dragging) can recover the original messages.

### Mathematical Explanation of Two-Time Pad Failure

In the One-Time Pad or stream cipher model, perfect secrecy requires the key (or keystream) to be random. the same lenght as the message, and never reused. The math below demonstrates why reuse is catastrophic:

Let K be the keystream generated by the encryption(Nonce||Counter).

Encryption 1: $C1 = P1 \oplus K$ Encryption 2: $C2 = P2 \oplus K$

When an adversary intercepts C1 and C2 they can compute their XOR sum (X): $X = C1 \oplus C2$ and then with substituting the encryption equations: $X = (P1 \oplus K) \oplus (P2 \oplus K)$. Due to the associative and commutative properties of XOR, we can rearrange the terms: $X = P1 \oplus P2 \oplus (K \oplus K)$ and then since any value XORed with itself is zero we get $X = P1 \oplus P2 \oplus 0$ $X = P1 \oplus P2$. The keystream (K) has been mathematically eliminated, if the attacker possesses a known plaintext (P1), they can immediately recover the second message (P2) by computing: $P2 = X \oplus P1$

---

## Task 5 — Birthday Attack on CTR (15 pts)

### Source Code

```
import os
import secrets
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

# --- Helper Functions for Encryption & XOR ---

def AESCTREncryption(key, NonceInteger, Plaintext):
    """Encrypts data using AES-CTR with a 16-bit nonce converted to bytes."""
    NonceBytes = NonceInteger.to_bytes(16, 'big')
    cipher = Cipher(algorithms.AES(key), modes.CTR(NonceBytes), backend=default_backend())
    Encryptor = cipher.encryptor()
    return Encryptor.update(Plaintext) + Encryptor.finalize()

def XORByter(b1, b2):
    """XORs two byte strings."""
    return bytes([x ^ y for x, y in zip(b1, b2)])

def DemonstrateTwoTimePad(NonceBits=16):
    """
    Simulates encryption to find a collision and proves C1^C2 == P1^P2.
    """
    print("\n" + "="*60)
    print("### Collision Enabling Two-Time Pad Demonstration")
    print("="*60)

    #Use a fixed key for this demonstration system
    SystemKey = os.urandom(32)
    History = {}
    MaxNonceValues = 2**NonceBits
    Trials = 0
```

```
    print(f"[*] Simulating encrypted traffic with {NonceBits}-bit nonces...")

    while True:
        Trials += 1
        #Generate Nonce
        Nonce = secrets.randbelow(MaxNonceValues)

        #Generate a random Plaintext (P2)
        Plaintext = f"SecretMsg_{Trials}_Random_{secrets.token_hex(4)}".encode()

        #Encrypt (C2)
        Ciphertext = AESCTREncryption(SystemKey, Nonce, Plaintext)

        #Check for Collision
        if Nonce in History:
            print(f"NONCE COLLISION FOUND after {Trials} encryptions!")
            print(f"    Nonce Value: {Nonce}")

            #Retrieve the previous message (P1, C1) that used this nonce
            OldCiphertext, OldPlaintext = History[Nonce]

            #Calculate XOR of Ciphertexts and XOR of Plaintexts
            XorCiphertexts = XORByter(OldCiphertext, Ciphertext) # C1 ^ C2
            XorPlaintexts  = XORByter(OldPlaintext, Plaintext)   # P1 ^ P2

            print(f"\n--- PROOF: C1^C2 == P1^P2 ---")
            print(f"P1 (Old): {OldPlaintext}")
            print(f"P2 (New): {Plaintext}")
            print("-" * 40)
            print(f"Calculated (C1 ^ C2): {XorCiphertexts.hex()}")
            print(f"Calculated (P1 ^ P2): {XorPlaintexts.hex()}")

            if XorCiphertexts == XorPlaintexts:
                print("\nThe mathematical relationship holds.")
                print("          Keystream reuse confirmed.")
            else:
                print("\nFailure has occured")

            break #Stop after finding the vulnerability

        #Store for collision checking
        History[Nonce] = (Ciphertext, Plaintext)

def CTRNonce(NonceBits=16):
    """
    Generates random nonces until a collision is found.
    Returns the number of times needed till a collision is found.
    """
    SeenNonces = set()
    Trials = 0
    MaxNonceValues = 2**NonceBits

    while True:
        Trials += 1
        #Generates a random integer acting as the nonce
        Nonce = secrets.randbelow(MaxNonceValues)

        if Nonce in SeenNonces:
            return Trials #Collision has occured

        SeenNonces.add(Nonce)


if __name__ == "__main__":
    NONCE_BITS = 16
```

```
NUM_EXPERIMENTS = 20
Results = []

print(f"Running {NUM_EXPERIMENTS} experiments with {NONCE_BITS}-bit nonces...")
print("-" * 40)

#Run the Statistical Experiments
for i in range(NUM_EXPERIMENTS):
    q = CTRNonce(NONCE_BITS)
    Results.append(q)
    print(f"Run {i+1}: Collision after {q} encryptions")

#Calculate the statistics
Average = sum(Results) / len(Results)
Approximation = 1.25 * (2**(NONCE_BITS / 2))

print("-" * 40)
print(f"Average collisions: {Average:.2f}")
print(f"Theoretical estimate:   {Approximation:.2f}")
print("-" * 40)

#Run the Demonstration
DemonstrateTwoTimePad(NONCE_BITS)
```

## Nonce Size Used

**Nonce Bits =** [16 bits] **Theoretical birthday bound:** q ≈ 1.25 × 2^(Nonce Bits/2) ≈ 320

## Collision Experiment Results (≥ 20 Runs)

| Run | Encryptions Until Collision |
|---|---|
| 1 | Collision after 106 encryptions |
| 2 | Collision after 274 encryptions |
| 3 | Collision after 68 encryptions |
| 4 | Collision after 325 encryptions |
| 5 | Collision after 478 encryptions |
| 6 | Collision after 581 encryptions |
| 7 | Collision after 303 encryptions |
| 8 | Collision after 371 encryptions |
| 9 | Collision after 182 encryptions |
| 10 | Collision after 128 encryptions |
| 11 | Collision after 170 encryptions |
| 12 | Collision after 563 encryptions |
| 13 | Collision after 504 encryptions |
| 14 | Collision after 705 encryptions |
| 15 | Collision after 331 encryptions |
| 16 | Collision after 517 encryptions |
| 17 | Collision after 383 encryptions |
| 18 | Collision after 383 encryptions |
| 19 | Collision after 302 encryptions |
| 20 | Collision after 161 encryptions |

**Average collision point:** 341.75

## Comparison to Theoretical Birthday Bound

| Metric | Value |
|---|---|
| r (nonce bits) | 16 |

| Metric | Value |
|---|---|
| Theoretical bound (1.2 × 2^(r/2)) | 320 |
| Observed average | 341.75 |
| Difference | 21.75 |

The observed average number of trials required to produce a collision was 341.75, this closely aligns with the theoretical birthday bound estimate of 320. The deviation of 21.75 (or 6.8%) is insignificant and is expected due to the nature of the task and the small sample size. Theoretically with more itterations the average would be much closer to the theoretical limit, this result confirms the Birthday Paradox as the collisions occurred in the hundreds range instead of the tens of thousands range demonstrating how quickly a small nonce space is exhausted.

### Collision Enabling Two-Time Pad Demonstration

```
┌──(kali㉿kali)-[~/Lab3/Task5]
└─$ python3 BirthdayAttack.py
Running 20 experiments with 16-bit nonces...
────────────────────────────────────────────
Run 1: Collision after 235 encryptions
Run 2: Collision after 383 encryptions
Run 3: Collision after 642 encryptions
Run 4: Collision after 559 encryptions
Run 5: Collision after 138 encryptions
Run 6: Collision after 86 encryptions
Run 7: Collision after 199 encryptions
Run 8: Collision after 371 encryptions
Run 9: Collision after 298 encryptions
Run 10: Collision after 366 encryptions
Run 11: Collision after 265 encryptions
Run 12: Collision after 340 encryptions
Run 13: Collision after 553 encryptions
Run 14: Collision after 666 encryptions
Run 15: Collision after 251 encryptions
Run 16: Collision after 149 encryptions
Run 17: Collision after 233 encryptions
Run 18: Collision after 390 encryptions
Run 19: Collision after 437 encryptions
Run 20: Collision after 455 encryptions
────────────────────────────────────────────
Average collisions: 350.80
Theoretical estimate:   320.00
────────────────────────────────────────────

═══════════════════════════════════════════════════════════════
### Collision Enabling Two-Time Pad Demonstration
═══════════════════════════════════════════════════════════════
[*] Simulating encrypted traffic with 16-bit nonces...
NONCE COLLISION FOUND after 423 encryptions!
    Nonce Value: 43119

── PROOF: C1^C2 == P1^P2 ──
P1 (Old): b'SecretMsg_69_Random_35efe96f'
P2 (New): b'SecretMsg_423_Random_fb8b1fde'
────────────────────────────────────────────
Calculated (C1 ^ C2): 00000000000000000000020b6c0d330f0a0b02326c53075e07085002
Calculated (P1 ^ P2): 00000000000000000000020b6c0d330f0a0b02326c53075e07085002

The mathematical relationship holds.
          Keystream reuse confirmed.
```

### Explanation: How Nonce Collision Leads to CTR Insecurity

The security of AES-CTR relies entirely on the uniqueness of the input block (Nonce + Counter) to ensure that the output keystream is unique for every encryption. As demonstrated in Task 5 as the number of our encrypted messages increases, the probability of generating a duplicate nonce increases according to the Birthday Paradox. If a nonce collision occurs while using the same key, the encryption system generates the exact same keystream for two different messages. This collision immediately recreates the conditions of the Two-Time Pad vulnerability mentioned in task 4. Once the collision occurs ($C1 \oplus C2 = P1 \oplus P2$) an attacker to bypass the AES encryption entirely and recover plaintext using simple XOR operations. Therefore, nonce uniqueness is not merely a best practice but a hard requirement as without it the confidentiality of CTR fails.

---

## Task 6 — Integrity vs. Confidentiality (AES-GCM) (10 pts)

### Source Code

```
"""
Task 6 — Integrity vs. Confidentiality (AES-GCM) (10 pts)
```

```
============================================================
Demonstrates the difference between confidentiality-only (CBC) and
authenticated encryption (GCM / AEAD).

CBC:
  - Flipping a byte in the ciphertext corrupts the corresponding plaintext block
    and causes predictable bit-flips in the NEXT block — but CBC accepts it silently.
  - No integrity guarantee: tampering is undetected.

GCM (AEAD):
  - The authentication tag T covers the entire ciphertext.
  - Any modification to the ciphertext OR the tag causes decryption to raise
    an InvalidTag exception — tampering is always detected.
"""

import os
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend
from cryptography.exceptions import InvalidTag

KEY_SIZE   = 32   # 256-bit
BLOCK_SIZE = 16
GCM_NONCE_SIZE = 12   # 96-bit recommended for GCM


# ---------------------------------------------------------------------------
# CBC helpers (PKCS#7)
# ---------------------------------------------------------------------------

def pkcs7_pad(data: bytes) -> bytes:
    padder = padding.PKCS7(BLOCK_SIZE * 8).padder()
    return padder.update(data) + padder.finalize()


def pkcs7_unpad(data: bytes) -> bytes:
    unpadder = padding.PKCS7(BLOCK_SIZE * 8).unpadder()
    return unpadder.update(data) + unpadder.finalize()


def cbc_encrypt(key: bytes, iv: bytes, plaintext: bytes) -> bytes:
    padded = pkcs7_pad(plaintext)
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    enc = cipher.encryptor()
    return enc.update(padded) + enc.finalize()


def cbc_decrypt(key: bytes, iv: bytes, ciphertext: bytes) -> bytes:
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    dec = cipher.decryptor()
    padded = dec.update(ciphertext) + dec.finalize()
    return pkcs7_unpad(padded)


def flip_byte(data: bytes, offset: int) -> bytes:
    """Flip a single byte in data at the given offset."""
    ba = bytearray(data)
    ba[offset] ^= 0xFF
    return bytes(ba)


# ---------------------------------------------------------------------------
# Task 6a: CBC tamper test
# ---------------------------------------------------------------------------
```

```python
def test_cbc_tamper(key: bytes, plaintext: bytes) -> None:
    print("=== CBC Tamper Test ===")
    iv = os.urandom(BLOCK_SIZE)
    ciphertext = cbc_encrypt(key, iv, plaintext)
    tampered   = flip_byte(ciphertext, offset=0)  # flip first byte

    print(f"  Original  ciphertext[0]: {ciphertext[0]:02x}")
    print(f"  Tampered  ciphertext[0]: {tampered[0]:02x}")

    try:
        recovered = cbc_decrypt(key, iv, tampered)
        print(f"  Decryption succeeded (no authentication check in CBC).")
        print(f"  Original  plaintext (hex): {plaintext[:32].hex()}")
        print(f"  Recovered plaintext (hex): {recovered[:32].hex()}")
        print(f"  Data corruption detected by comparison: {plaintext[:32] != recovered[:32]}")
    except Exception as e:
        print(f"  Decryption raised: {e}")

    print()


# ---------------------------------------------------------------------------
# Task 6b: GCM tamper test
# ---------------------------------------------------------------------------

def test_gcm_tamper(key: bytes, plaintext: bytes) -> None:
    print("=== GCM Tamper Test ===")
    aesgcm = AESGCM(key)
    nonce  = os.urandom(GCM_NONCE_SIZE)
    ciphertext_tag = aesgcm.encrypt(nonce, plaintext, associated_data=None)
    # ciphertext_tag = ciphertext || 16-byte tag

    tampered = flip_byte(ciphertext_tag, offset=0)  # flip one byte in ciphertext

    print(f"  Original  ciphertext_tag[0]: {ciphertext_tag[0]:02x}")
    print(f"  Tampered  ciphertext_tag[0]: {tampered[0]:02x}")

    try:
        aesgcm.decrypt(nonce, tampered, associated_data=None)
        print("  ERROR: Decryption succeeded despite tampering — this should not happen.")
    except InvalidTag:
        print("  InvalidTag exception raised — tampering detected. (expected)")
    except Exception as e:
        print(f"  Unexpected exception: {e}")

    print()


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    key       = os.urandom(KEY_SIZE)
    plaintext = b"This is a secret message that must not be tampered with!" * 2

    print(f"Key (hex): {key.hex()}\n")

    test_cbc_tamper(key, plaintext)
    test_gcm_tamper(key, plaintext)

    print("=== Summary ===")
    print("  CBC: accepts tampered ciphertext — corrupts plaintext silently (no integrity).")
    print("  GCM: rejects tampered ciphertext — raises InvalidTag (AEAD = confidentiality + integrity).")
```

### CBC Tamper Evidence

```
Key (hex): 63216b0313b320049e89c24460cbc249921bd6a1449796396c950ecec768786e

=== CBC Tamper Test ===
  Original  ciphertext[0]: ba
  Tampered  ciphertext[0]: 45
  Decryption succeeded (no authentication check in CBC).
  Original  plaintext (hex): 546869732069732061207365637265742064657373616765207468617420d75
  Recovered plaintext (hex): 5963f5dbe9e62868bc156abdb4f7dceddf6d657373616765207468617420d75
  Data corruption detected by comparison: True
```

```
=== CBC Tamper Test ===
  Original  ciphertext[0]: ba
  Tampered  ciphertext[0]: 45
  Decryption succeeded (no authentication check in CBC).
  Original  plaintext (hex): 546869732069732061207365637265742064657373616765207468617420d75
  Recovered plaintext (hex): 5963f5dbe9e62868bc156abdb4f7dceddf6d657373616765207468617420d75
  Data corruption detected by comparison: True
```

**Observation:** CBC accepted the tampered ciphertext and still decrypted, producing corrupted plaintext without any authentication error. This shows CBC provides confidentiality but not integrity.

### GCM Tamper Evidence

[INSERT SCREENSHOT: Flip 1 byte in GCM ciphertext or tag → decryption fails with authentication error]

```
=== GCM Tamper Test ===
  Original  ciphertext_tag[0]: 52
  Tampered  ciphertext_tag[0]: ad
  InvalidTag exception raised — tampering detected. (expected)
```

```
=== GCM Tamper Test ===
  Original  ciphertext_tag[0]: 52
  Tampered  ciphertext_tag[0]: ad
  InvalidTag exception raised — tampering detected. (expected)
```

**Observation:** AES-GCM rejected the tampered ciphertext by failing authentication and raising an InvalidTag error. No plaintext was returned, demonstrating integrity protection.

### Comparison: Confidentiality vs. Integrity

Confidentiality ensures that an attacker cannot learn the plaintext without the secret key. Integrity ensures that an attacker cannot modify the ciphertext without the receiver detecting that modification.

In AES-CBC mode, encryption provides confidentiality by transforming plaintext blocks into ciphertext blocks. However, CBC does not authenticate the ciphertext. As demonstrated above, flipping a single byte in the ciphertext caused the first plaintext block to become corrupted, yet decryption still succeeded without any error. This shows that CBC does not provide integrity — tampering may go undetected and result in corrupted but accepted plaintext.

In contrast, AES-GCM is an AEAD (Authenticated Encryption with Associated Data) mode. It produces both ciphertext and an authentication tag T that covers the entire ciphertext (and any associated data). During decryption, this tag is verified. If any bit of the ciphertext or tag is modified, decryption fails and returns ⊥ (represented here by an InvalidTag exception). Therefore, AES-GCM provides both confidentiality and integrity.

In practice, encryption without authentication (such as CBC alone) is insufficient for secure systems. Modern cryptographic standards recommend using AEAD modes like AES-GCM to ensure both secrecy and tamper detection.

---

## Task 7 — Performance Benchmarking (10 pts)

### Source Code

```
"""
Task 7 — Performance Benchmarking (10 pts)
==========================================
Benchmarks AES-ECB, AES-CBC, AES-CTR, and AES-GCM across three file sizes.

Methodology:
```

```
- File sizes: 1 KB, 1 MB, 10 MB
- Modes: ECB, CBC, CTR, GCM
- Each (mode, file size) combination is repeated 5 times and averaged
  to reduce timing variance caused by system load and cache effects.
- Reports average encryption and decryption time in seconds.
"""

import os
import time
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend

KEY_SIZE      = 32    # 256-bit
BLOCK_SIZE    = 16
GCM_NONCE_SIZE = 12
REPETITIONS   = 5     # repeat 5 times and average

FILE_SIZES = {
    "1 KB":  1 * 1024,
    "1 MB":  1 * 1024 * 1024,
    "10 MB": 10 * 1024 * 1024,
}


# ---------------------------------------------------------------------------
# Padding helpers (ECB and CBC need block-aligned input)
# ---------------------------------------------------------------------------

def pkcs7_pad(data: bytes) -> bytes:
    padder = padding.PKCS7(BLOCK_SIZE * 8).padder()
    return padder.update(data) + padder.finalize()


def pkcs7_unpad(data: bytes) -> bytes:
    unpadder = padding.PKCS7(BLOCK_SIZE * 8).unpadder()
    return unpadder.update(data) + unpadder.finalize()


# ---------------------------------------------------------------------------
# Encrypt / decrypt functions (one call per mode)
# ---------------------------------------------------------------------------

def bench_ecb(key: bytes, plaintext: bytes) -> tuple[float, float]:
    """Return (enc_time, dec_time) for AES-ECB."""
    padded = pkcs7_pad(plaintext)

    t0 = time.perf_counter()
    cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=default_backend())
    enc = cipher.encryptor()
    ct = enc.update(padded) + enc.finalize()
    enc_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    cipher = Cipher(algorithms.AES(key), modes.ECB(), backend=default_backend())
    dec = cipher.decryptor()
    pkcs7_unpad(dec.update(ct) + dec.finalize())
    dec_time = time.perf_counter() - t0

    return enc_time, dec_time


def bench_cbc(key: bytes, plaintext: bytes) -> tuple[float, float]:
    """Return (enc_time, dec_time) for AES-CBC with PKCS#7."""
    iv     = os.urandom(BLOCK_SIZE)
```

```
    padded = pkcs7_pad(plaintext)

    t0 = time.perf_counter()
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    enc = cipher.encryptor()
    ct = enc.update(padded) + enc.finalize()
    enc_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
    dec = cipher.decryptor()
    pkcs7_unpad(dec.update(ct) + dec.finalize())
    dec_time = time.perf_counter() - t0

    return enc_time, dec_time


def bench_ctr(key: bytes, plaintext: bytes) -> tuple[float, float]:
    """Return (enc_time, dec_time) for AES-CTR."""
    nonce = os.urandom(BLOCK_SIZE)

    t0 = time.perf_counter()
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce), backend=default_backend())
    enc = cipher.encryptor()
    ct = enc.update(plaintext) + enc.finalize()
    enc_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce), backend=default_backend())
    dec = cipher.decryptor()
    dec.update(ct) + dec.finalize()
    dec_time = time.perf_counter() - t0

    return enc_time, dec_time


def bench_gcm(key: bytes, plaintext: bytes) -> tuple[float, float]:
    """Return (enc_time, dec_time) for AES-GCM (AEAD)."""
    aesgcm = AESGCM(key)
    nonce  = os.urandom(GCM_NONCE_SIZE)

    t0 = time.perf_counter()
    ct = aesgcm.encrypt(nonce, plaintext, associated_data=None)
    enc_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    aesgcm.decrypt(nonce, ct, associated_data=None)
    dec_time = time.perf_counter() - t0

    return enc_time, dec_time


BENCH_FUNCS = {
    "ECB": bench_ecb,
    "CBC": bench_cbc,
    "CTR": bench_ctr,
    "GCM": bench_gcm,
}


# ---------------------------------------------------------------------------
# Runner — each combination repeated 5 times and averaged
# ---------------------------------------------------------------------------

def run_benchmark(key: bytes) -> dict:
    results = {}
```

```
    for size_label, size_bytes in FILE_SIZES.items():
        plaintext = os.urandom(size_bytes)
        for mode_name, bench_fn in BENCH_FUNCS.items():
            enc_times = []
            dec_times = []
            for _ in range(REPETITIONS):
                e, d = bench_fn(key, plaintext)
                enc_times.append(e)
                dec_times.append(d)
            avg_enc = sum(enc_times) / REPETITIONS
            avg_dec = sum(dec_times) / REPETITIONS
            results[(size_label, mode_name)] = (avg_enc, avg_dec)
    return results


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    key = os.urandom(KEY_SIZE)
    print(f"=== Task 7: AES Mode Performance Benchmark ===")
    print(f"Key (hex): {key.hex()}")
    print(f"Repetitions per combination: {REPETITIONS} (results are averaged)\n")

    results = run_benchmark(key)

    # Print results table
    header = f"{'File Size':<10} {'Mode':<6} {'Avg Enc (s)':<16} {'Avg Dec (s)'}"
    print(header)
    print("-" * len(header))
    for size_label in FILE_SIZES:
        for mode_name in BENCH_FUNCS:
            avg_enc, avg_dec = results[(size_label, mode_name)]
            print(f"{size_label:<10} {mode_name:<6} {avg_enc:<16.6f} {avg_dec:.6f}")
        print()
```

## Results Table

| File Size | Mode | Avg Enc Time (s) | Avg Dec Time (s) |
|---|---|---|---|
| 1 KB | ECB | 0.000702 | 0.000007 |
| 1 KB | CBC | 0.000005 | 0.000004 |
| 1 KB | CTR | 0.000005 | 0.000004 |
| 1 KB | GCM | 0.000002 | 0.000001 |
| 1 MB | ECB | 0.000535 | 0.000516 |
| 1 MB | CBC | 0.000996 | 0.000475 |
| 1 MB | CTR | 0.000391 | 0.000264 |
| 1 MB | GCM | 0.000253 | 0.000279 |
| 10 MB | ECB | 0.002973 | 0.004152 |
| 10 MB | CBC | 0.007460 | 0.004016 |
| 10 MB | CTR | 0.002907 | 0.002410 |
| 10 MB | GCM | 0.002125 | 0.002226 |

## Performance Analysis

PARAGRAPH 1 — DISCUSS: CBC encryption is sequential because each ciphertext block depends on the previous ciphertext block ($C_i = E_k(P_i \oplus C_{i-1})$), which prevents parallelization during encryption. This is reflected in the results: for 10 MB, CBC encryption (0.007460 s) is noticeably slower than CTR (0.002907 s) and GCM (0.002125 s). In contrast, CTR and GCM generate keystream blocks independently using $E_k(\text{Nonce} \| \text{Counter}_i)$, allowing full parallelization. ECB is also fully parallelizable and appears relatively fast, but it is insecure in practice due to deterministic pattern leakage.

PARAGRAPH 2 — DISCUSS: AES-GCM introduces authentication overhead by computing a GHASH over the ciphertext to produce an authentication tag. Despite this additional computation, GCM performance is comparable to CTR and often slightly faster in these results (e.g., 10 MB encryption: GCM 0.002125 s vs CTR 0.002907 s). This demonstrates that the integrity protection provided by AEAD comes with minimal performance cost, making AES-GCM the preferred modern mode for secure systems.

---

# Key Lessons Learned

ECB and Determinism: ECB mode is deterministic: identical plaintext blocks always produce identical ciphertext blocks ($C_i = E_k(P_i)$). This leaks structural patterns in the data and allows an adversary to distinguish encryptions, violating semantic security (IND-CPA). Even though ECB may appear fast in benchmarks, it is insecure for real-world use due to pattern leakage.

IV / Nonce Freshness in CBC and CTR: Both CBC and CTR rely on randomness (IV or nonce) to achieve semantic security. If a fresh, unpredictable IV/nonce is used for each encryption, ciphertexts remain unlinkable. Reusing an IV or nonce under the same key leaks information about message structure and breaks security assumptions.

Nonce Reuse and Two-Time Pad Failure in CTR: In CTR mode, ciphertext is generated as $C = P \oplus K$ where K is the keystream derived from the nonce and counter. If the same nonce is reused with the same key, the identical keystream is reused. XORing two ciphertexts eliminates the keystream: $C_1 \oplus C_2 = P_1 \oplus P_2$. This creates a two-time pad scenario, allowing plaintext recovery via crib dragging. Nonce uniqueness is therefore a strict security requirement.

Birthday Bound and Nonce Space Sizing: The birthday paradox shows that collisions occur after approximately $1.2 \times 2^{(r/2)}$ encryptions for an r-bit nonce. If the nonce space is too small, collisions become likely, leading to keystream reuse and catastrophic failure in CTR. This demonstrates why modern systems use large nonce sizes (e.g., 96 bits in GCM).

AEAD and Why Confidentiality Alone Is Insufficient: Encryption alone does not guarantee integrity. As demonstrated with CBC, tampered ciphertext may decrypt to corrupted plaintext without detection. AEAD modes such as AES-GCM provide both confidentiality and integrity by generating an authentication tag. Any modification to the ciphertext or tag causes decryption to fail, preventing silent corruption.

Performance Trade-Offs Between Modes: ECB, CTR, and GCM can be parallelized and scale efficiently with large data sizes. CBC encryption is sequential due to block chaining, making it slower at scale. Although GCM introduces authentication overhead (GHASH computation), benchmark results show that the performance cost is minimal compared to CTR. Given its integrity guarantees and competitive performance, AES-GCM is the preferred modern mode.

---

# Appendix

## Full Script Listings

[OPTIONAL: Include complete script code if not fully shown in task sections above]

## Additional Screenshots

[OPTIONAL: Include any additional supporting terminal output or evidence]